

Yuva Intern AI Pioneers Internship(Machine Learning Internship)

Week 1: Python for Machine Learning & Data Preprocessing

Name: Manasvi Beri

Tools used : Google Colab, Python, NumPy, Pandas, Matplotlib, Seaborn, Scikit-learn

1. Introduction

Data preprocessing is an important step in the machine learning workflow. Real world datasets often contain missing values, unnecessary features, noise, categorical variables, duplicate records, and numerical features with different scales.

In order to produce a dataset which is efficient to use in training and testing, cleaning and preprocessing of the dataset is necessary.

The objective of this project is to understand and implement fundamental data preprocessing techniques using Python. Here, The Titanic dataset was selected as the sample dataset for this project.

The dataset was explored, cleaned, analysed, and transformed into a format suitable for further machine learning applications.

2. Objectives

The main objectives of this project were:

1. To understand the fundamentals of machine learning using Python.
2. To learn how to load and inspect a dataset using Pandas and NumPy.
3. To identify and handle missing values.
4. To check for duplicate records.
5. To perform feature selection.
6. To encode categorical variables.
7. To perform exploratory data analysis.

8. To apply feature scaling to numerical variables.
9. To produce a clean and preprocessed dataset.
10. To document each preprocessing step.

3. Dataset Description

The **Titanic dataset** contains information about passengers who travelled on the Titanic.

The original dataset used in this project contained:

- **418 rows**
- **12 columns**

The original columns were:

Column	Description
Passenger ID	Unique passenger identification number
Survived	Survival status of the passenger
Pclass	Passenger class
Name	Passenger name
Sex	Gender of the passenger
Age	Age of the passenger
SibSp	Number of siblings/spouses aboard
Parch	Number of parents/children aboard
Ticket	Ticket number
Fare	Passenger fare
Cabin	Cabin information
Embarked	Port of embarkation

The Survived column indicates whether a passenger survived:

- 0 – Did not survive
- 1 – Survived

4. Technologies and Libraries Used

The following technologies and Python libraries were used:

Python

Used as the main programming language for implementing the preprocessing workflow.

NumPy

Used for numerical operations and working with numerical data.

Pandas

Used for loading, exploring, cleaning, and transforming the dataset.

Matplotlib

Used to create data visualizations.

Seaborn

Used to create statistical visualizations and plots.

Scikit-learn

Used for feature scaling using StandardScaler.

5. Loading the Dataset

The dataset was loaded into a Pandas DataFrame using the `read_csv()` function.

```
import pandas as pd
import numpy as np
```

```
df = pd.read_csv('/content/archive (5) (1).csv')
```

The first few records were displayed using:

```
df.head()
```

This helped verify that the dataset was loaded correctly.

6. Dataset Exploration

The dataset was examined using several Pandas functions.

Dataset dimensions

```
df.shape
```

The original dataset contained **418 rows and 12 columns**.

Column names

```
df.columns.tolist()
```

Dataset information

```
df.info()
```

This was used to understand the data types and number of non-null values in each column.

Statistical summary

`df.describe()` : The `describe()` function provided statistical information such as count, mean, standard deviation, minimum, maximum, and quartile values for numerical features.

7. Identification of Missing Values

Missing values were identified using:

```
df.isnull().sum()
```

The initial missing-value analysis showed:

Column	Missing Values
PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	86
SibSp	0
Parch	0
Ticket	0
Fare	1
Cabin	327
Embarked	0

The Age, Fare, and Cabin columns required preprocessing.

8. Handling Missing Values

8.1 Age

The Age column contained **86 missing values**.

The median age was used to replace the missing values:

```
df['Age'] = df['Age'].fillna(df['Age'].median())
```

The median was selected because it is less affected by extreme values than the mean.

8.2 Fare

The Fare column contained **one missing value**.

The missing value was replaced using the median:

```
df['Fare'] = df['Fare'].fillna(df['Fare'].median())
```

8.3 Cabin

The Cabin column contained **327 missing values out of 418 records**.

Because a very large proportion of this column was missing, it was removed from the dataset:

```
df = df.drop('Cabin', axis=1)
```

After handling the missing values, the dataset was checked again:

```
df.isnull().sum()
```

The remaining columns contained **zero missing values**.

9. Checking Duplicate Records

Duplicate records were checked using:

```
df.duplicated().sum()
```

This step was performed to ensure that repeated observations did not affect the analysis.

If your output showed 0 duplicates, write:

No duplicate records were found in the dataset.

If your output showed duplicates, replace that sentence with:

Duplicate records were removed using the drop_duplicates() method.

10. Feature Selection

Some columns were removed because they were not required for the preprocessing exercise.

The following columns were removed:

```
df = df.drop(['PassengerId', 'Name', 'Ticket'], axis=1)
```

Reasons for removal

PassengerId:

This column is simply an identifier and does not provide meaningful information for the analysis.

Name:

This is a text-based column with many unique values and was not required for this preprocessing task.

Ticket:

This column contains ticket identifiers and was not used for the current analysis.

After feature selection, the dataset contained the following features:

Survived

Pclass

Sex

Age

SibSp

Parch

Fare

Embarked

11. Exploratory Data Analysis

Exploratory Data Analysis (EDA) was performed to understand the characteristics and patterns in the dataset.

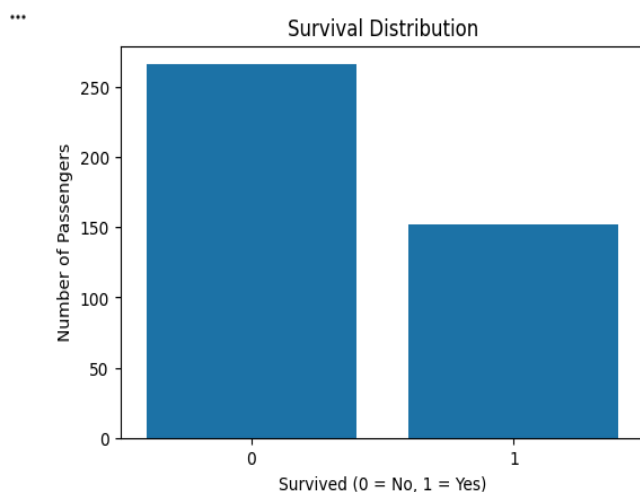
Several visualizations were created using Matplotlib and Seaborn.

11.1 Survival Distribution

A count plot was created to visualize the number of passengers who survived and did not survive.

```
sns.countplot(x='Survived', data=df)
```

```
plt.figure(figsize=(6, 4))
sns.countplot(x='Survived', data=df)
plt.title('Survival Distribution')
plt.xlabel('Survived (0 = No, 1 = Yes)')
plt.ylabel('Number of Passengers')
plt.show()
```



Observation

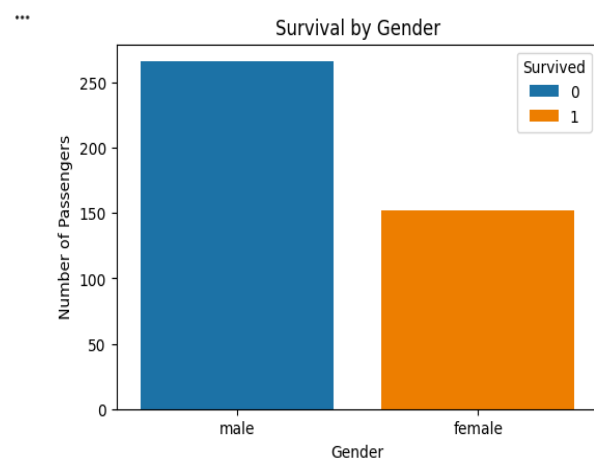
The graph shows the distribution of passengers according to their survival status.

11.2 Survival by Gender

A count plot was used to examine survival distribution by gender.

```
sns.countplot(x='Sex', hue='Survived', data=df)
```

```
plt.figure(figsize=(6, 4))
sns.countplot(x='Sex', hue='Survived', data=df)
plt.title('Survival by Gender')
plt.xlabel('Gender')
plt.ylabel('Number of Passengers')
plt.show()
```



Observation

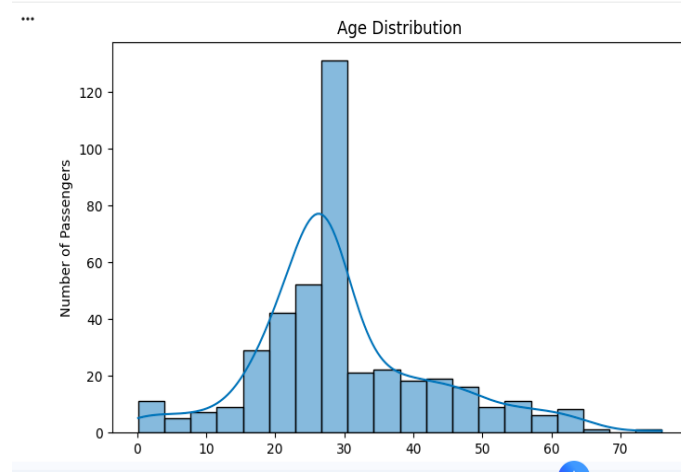
The visualization allows comparison of survival counts across the gender categories.

11.3 Age Distribution

A histogram was created to examine the distribution of passenger ages.

```
sns.histplot(df['Age'], bins=20, kde=True)
```

```
plt.figure(figsize=(8, 5))
sns.histplot(df['Age'], bins=20, kde=True)
plt.title('Age Distribution')
plt.xlabel('Age')
plt.ylabel('Number of Passengers')
plt.show()
```



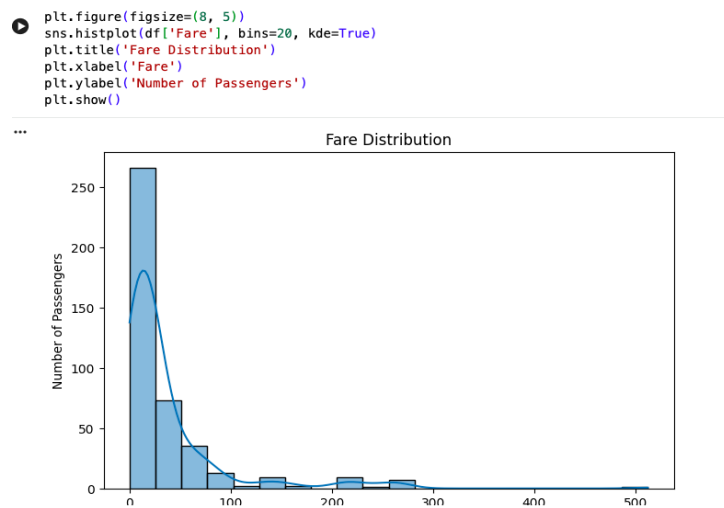
Observation

The histogram shows how passenger ages are distributed throughout the dataset.

11.4 Fare Distribution

The distribution of passenger fares was visualized using a histogram.

```
sns.histplot(df['Fare'], bins=20, kde=True)
```



Observation

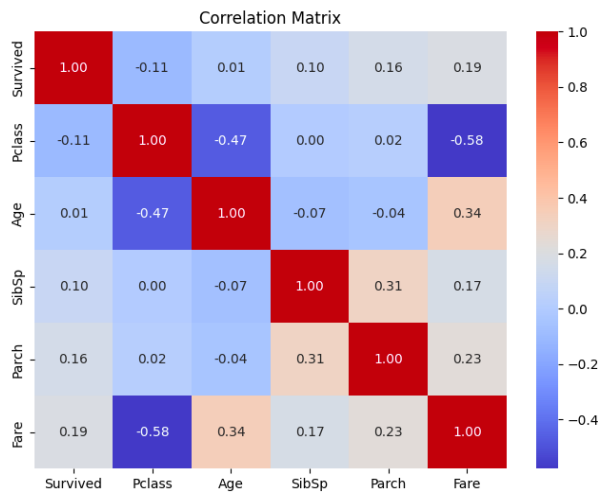
The visualization shows the distribution of fares paid by passengers.

11.5 Correlation Analysis

A correlation matrix was created for numerical variables.

```
correlation = df.select_dtypes(include=np.number).corr()
```

```
sns.heatmap(correlation, annot=True, cmap='coolwarm', fmt='.2f')
```



Observation

The correlation matrix provides an overview of the relationships between numerical variables in the dataset.

12. Encoding Categorical Variables

The dataset contained categorical variables such as:

- Sex
- Embarked

Machine learning algorithms generally require numerical input, so these categorical variables were converted into numerical representations.

The unique categories were first examined:

```
df['Sex'].unique()
df['Embarked'].unique()
```

One-hot encoding was then performed:

```
df = pd.get_dummies(
    df,
    columns=['Sex',
              'Embarked'],
    drop_first=True
)
```

The resulting categorical features were converted into numerical values.

The final encoded columns included:

```
Sex_male  
Embarked_Q  
Embarked_S
```

The Boolean values were converted to integers:

```
df = df.astype(int)
```

This resulted in a completely numerical dataset.

13. Feature Scaling

Numerical features can have different ranges. For example, Age and Fare may have considerably different numerical scales.

To address this, **StandardScaler** from Scikit-learn was used.

```
from sklearn.preprocessing import StandardScaler  
  
scaler = StandardScaler()  
  
df[['Age', 'Fare']] = scaler.fit_transform(  
    df[['Age', 'Fare']])
```

Standardization transforms the selected numerical features to approximately:

- Mean = 0
- Standard deviation = 1

This makes the numerical features more comparable in scale and prepares them for further machine learning applications.

14. Final Pre processed Dataset

After completing the preprocessing steps, the final dataset contained the following columns:

Survived
Pclass
Age
SibSp
Parch
Fare
Sex_male
Embarked_Q
Embarked_S

The dataset was checked again for missing values:

```
df.isnull().sum()
```

All remaining columns contained **zero missing values**.

The final dataset was saved as:

```
df.to_csv('/content/cleaned_titanic_dataset.csv', index=False)
```

The resulting file was named:

cleaned_titanic_dataset.csv

16. Conclusion

The Titanic dataset was successfully cleaned and preprocessed using Python and commonly used data science libraries.

This task provided practical experience with data loading, data exploration, missing value handling, feature selection, exploratory data analysis, categorical variable encoding, and numerical feature scaling.

After preprocessing, the dataset contained no missing values in the retained features, categorical variables had been converted into numerical representations, and selected numerical features had been standardized.

The final cleaned dataset was saved as a CSV file and the complete preprocessing workflow was documented in the Google Collab notebook.

This project helped build a foundation in Python-based data preprocessing and prepared the dataset for further machine learning analysis.

17. Task Files

The project contains the following files in gituhub.

1. YuvaIntern_Week1_Data_Preprocessing.ipynb

Contains the complete Python implementation and preprocessing workflow.

2. cleaned_titanic_dataset.csv

Contains the final cleaned and pre processed dataset.

3. Week1_Data_Preprocessing_Report.pdf

Contains the documentation of the preprocessing steps and observations.

4. README.md

Provides an overview of the project and its contents.

18. GitHub Repository

GitHub Repository:

<https://github.com/manasviberi/Yuva-intern-AI-Pioneers-internship-ML-week-1>